

Prepare for Microsoft in 90 Days

Prepare around the actual role: problem solving, design, coding, testing, and relevant technical depth. A storage role, product backend role, and ML platform role need different specialization. Use the job description and recruiter guidance to select that depth.

For: Engineers who can already write basic programs and want a realistic, evidence-driven preparation plan. This is a preparation timeline, not a promise of a Microsoft offer within three months.

Suggested pace: 12 weeks, 8-12 focused hours per week | Early-career to experienced SWE

What you will be able to do

- Solve unseen coding problems with a stated invariant, correct implementation, tests, and complexity.
- Explain one production-style design and the failure scenarios that shape it.
- Discuss your own project decisions, contribution, results, and limitations without invented metrics.
- Use mock feedback to decide what to revise instead of measuring readiness only by question counts.

Build the right mental model

Start from a diagnostic

Attempt one medium coding problem without notes, explain a previous project for ten minutes, and design a small API. Record whether the gaps are recognition, implementation, testing, communication, or architecture. The weakest repeatable skill sets the next week's work.

Use official scope, not question predictions

Microsoft's technical guidance emphasizes problem solving, design, coding, and testing, with role-dependent areas such as distributed systems, networking, and AI/ML. Confirm the level, round format, and allowed tools with the recruiter; no public list can predict your exact questions.

Keep preparation sustainable

Reserve short weekday sessions and one longer weekend mock. Keep one implementation language throughout the plan. Maintain an error log with the missed assumption, the correction, and a re-solve date; avoid restarting a new course every time a topic feels difficult.

Study with evidence.

Spend roughly half your time on coding and testing, one quarter on design or domain depth, and one quarter on projects, behavioral practice, and review. Adjust this split to the role and your diagnostic rather than treating it as a Microsoft hiring formula.

Weeks 1-3: Establish coding reliability

Practice arrays, hashing, two pointers, sliding windows, stacks, and binary search. For each problem, state constraints and a baseline, explain an invariant, implement, and test boundaries. Re-solve failed problems without looking at the original answer.

Deliverable and check: A small error log, six independent re-solves, and two timed mixed sets. Check that you can explain why the algorithm works, not only reproduce code.

Weeks 4-6: Broaden patterns and system basics

Add trees, graph traversal, heaps, intervals, and introductory dynamic programming. In parallel, review HTTP, TCP, processes, threads, memory, SQL indexes, and transactions through small experiments rather than reading alone.

Deliverable and check: One graph lab, one database lab, and a project architecture sketch with clear data ownership. Explain at least three failure cases.

Weeks 7-9: Practice role-relevant design

Work through a cache, notification service, file service, or other role-relevant system. Define APIs and data models, estimate load, then analyze consistency, retries, backpressure, and observability. Prepare truthful ownership, failure, and collaboration stories.

Deliverable and check: Three design walkthroughs and six concise story outlines. Every claimed result should be supportable and every major component should solve a named requirement.

Weeks 10-12: Simulate and close gaps

Alternate unseen coding mocks, project deep dives, and design discussions. Review the actual job requirements and current recruiter instructions. Use the last week for targeted correction, interview logistics, and rest rather than a flood of unfamiliar problems.

Deliverable and check: A mock scorecard showing improvement in weak dimensions; a verified environment and timezone; a short final revision list rather than a memorized script.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Turn a failed mock into a seven-day improvement cycle

You recognize a sliding-window problem, but your implementation returns the wrong result when characters repeat near the left boundary. You also run out of time before testing. The useful goal is fixing this failure pattern, not adding twenty unrelated problems.

A worked approach

1. Write the exact failing input and explain which invariant broke. For a unique-character window, the left boundary must never move backward and every character inside the current window must satisfy the chosen uniqueness representation.
2. Implement the correction from a blank editor. Trace repeated characters, an empty input if allowed, and a one-character input. Explain the cost of both time and the stored character state.
3. On days two and four, solve related but different window problems. Decide whether the window validity is monotone before selecting the pattern; arbitrary negative sums may require a different technique.
4. On day seven, run another unseen timed mock and reserve a testing segment. Compare clarification, correctness, testing, and explanation with the first scorecard rather than only elapsed time.

Expected result

A documented failure, a minimal correction, two transfer exercises, and a new mock score provide evidence of learning. Remembering the original solution is not enough to demonstrate transfer.

Verify the result

- Can you state the invariant before typing code?
- Does the boundary case fail on the original implementation and pass after the correction?
- Can you apply the idea to an unseen variation without reading a template?

Extend the exercise

Apply the same cycle to a design mistake: for example, a retry that can charge a customer twice. Identify the missing invariant, design an idempotency boundary, and test timeout and duplicate-request scenarios.

Make the learning stick.

Is 90 days enough?

It can be a useful preparation window if you already have foundations and consistent time. Readiness depends on the role and baseline. Use independent mocks and concrete skill checks; do not confuse a calendar deadline with a hiring guarantee.

Should I focus only on Microsoft-tagged questions?

No. Use tags as a way to organize practice, not as verified question predictions. Transferable algorithms, tests, design reasoning, and the actual role requirements are more dependable than alleged question frequency.

What if I do not know a technical answer?

State the known constraints, identify the uncertain assumption, and reason from fundamentals. Explain how you would verify it. Fabricated benchmark numbers or unsupported claims about production systems reduce credibility.

Avoid these traps

- Collecting solved counts while avoiding timed unseen problems and boundary tests.
- Treating every Microsoft team as if it had the same interview and specialization.
- Using confidential workplace code, data, or incidents in a public preparation project.

Ready for the next step?

- Recent unseen coding mocks show independent correctness and clear tests.
- One role-relevant design includes recovery, trade-offs, and capacity assumptions.
- Project and behavioral examples distinguish your actions from the team's work.
- Recruiter instructions, interview tools, timezone, and accommodations are confirmed.

Reliable next reads

- [Microsoft: technical interviewing](#)
- [MIT OpenCourseWare: algorithms](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.